

Contents

hide

Beginning

- 2. Calculating the energy and magnetisation
- Getting the files for the experiment
- Modifying the files
- A note about Python Classes
- The Constructor
- TASK 2a:
- Testing the files
- TASK 2b:

Appearance

hide

Text

- Small

Standard

Large
- Width
- Standard

Wide

Programming a 2D Ising Model/Calculating the energy and magnetisation

Page

Discussion

Read

View source

View history

Tools

This is the second section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 1. Introduction to the Ising model

Next section: 3. Introduction to Monte Carlo simulation

2. Calculating the energy and magnetisation

Getting the files for the experiment

Various scripts have been prepared to assist you with this experiment, and you can obtain them by downloading the files from the Blackboard.

We recommend you open the extracted folder with JupyterLab, which has an integrated terminal where you can run the simulation scripts.

Modifying the files

The file `IsingLattice.py` contains a Python class called `IsingLattice`. This class will be the basis for our model of the system, and is restricted to modelling the two dimensional case. It contains a number of stub methods that you will complete as you work through the experiment.

A note about Python Classes

The file `IsingLattice.py` makes use of a Python feature called *class*. You should be familiar with the idea that all Python *objects* have an associated *type* -- you have already encountered objects such as integers, string, lists, and NumPy arrays. Classes allow us to define new types of object. In this case, we define a type called `IsingLattice`:

```
class IsingLattice:
```

Each class is allowed to have certain attributes and functions (known as methods) associated with it. We can see, for example, that each `IsingLattice` object has the methods `energy()`, `magnetisation()`, `montecarlostep(T)`, and `statistics()` (the method `__init__` will be explained shortly). Each of these methods takes a special first argument called `self`, which is a variable pointing to the current copy of `IsingLattice`. To use the new `IsingLattice` object that we have defined, we use code like:

```
import IsingLattice

il = IsingLattice(5, 5) #create an IsingLattice object with 5 row and 5 columns

energy = il.energy() #call the energy() method of our IsingLattice object
```

When writing code "within" the class definition, all attributes and methods belonging to the class must be accessed using the "self." notation. You can see an example of this in the `montecarlostep()` method:

```
energy = self.energy()
```

This line creates a variable called `energy` which contains whatever value was returned by the method `energy()` which is part of the `IsingLattice` class. If you just want to define a local variable instead, like an iterator in a loop, the `self.` notation is not required.

```
def magnetisation(self):
    # we have to use self.lattice, because the lattice variable is part of the object, not
    local to this function
    for i in self.lattice:
        ...
        # we use i, not self.i
        # we don't need to access i outside this function
```

Have a look at the other files that you have been given to see how this `IsingLattice` object is used.

The Constructor

The only method currently filled in is `__init__`. This is a special method called a *constructor*. Whenever a new `IsingLattice` object is created, using code like `il = IsingLattice(5,5)`, this method is called automatically. It takes two arguments (excluding "self"), which are simply the number of rows and columns that our lattice will have. The attribute `self.lattice` is then created, which is a NumPy array with the requested number of rows and columns. The function `np.random.choice` is used to create a random collection of spins for our initial configuration. You can see the documentation for the function [numpy.random.choice](#).

TASK 2a:

Complete the functions `energy()` and `magnetisation()`, which should return the energy of the lattice and the total magnetisation, respectively. In the `energy()` function you may assume that $J=1.0$ at all times (in fact, we are working in *reduced units* in which all energies are a multiple of J , but there will be more information about this in later sections). Do not worry about the efficiency of the code at the moment — we will address the speed in a later part of the experiment. Paste your code in your report and describe how it works.

Testing the files

When you have completed the `energy()` and `magnetisation()` methods, we need to test that they work correctly. We have provided you with a script, `ILcheck.py`, which will create three `IsingLattice` objects and check their energies and magnetisations, displaying these on a graph. One configuration corresponds to the energy minimum, one to the energy maximum, and one to an intermediate state.

TASK 2b:

Run the `ILcheck.py` script from the a terminal using the command

```
python ILcheck.py
```

The displayed window has a series of control buttons in the bottom left, one of which will allow you to export the figure as a SVG or PNG image. Save an image of the `ILcheck.py` output, and include it in your report.

There are also some unit tests available in the `test_energy.py` file. These tests are designed to be run by simply running the `pytest` command in your terminal,

```
pytest
```

You should make sure your code passes the tests for the `energy()` and `magnetisation()` methods at this time. The file contains tests for methods that will only be implemented later on, so some failures are expected for now. These sorts of test are useful because they are easier to write and faster to run and can be made to run automatically.

You can also debug and test your program on a Jupyter notebook or IPython console by importing your `IsingLattice` class, creating one object, and inspecting its properties. If you do this, we recommend that you start your session running the commands:

```
%load_ext autoreload
%autoreload 2
```

This will ensure that when you try executing your code, you always run the most recent version (please note this isn't fool proof and if you find problems you may need to restart the kernel)!

This is the second section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 1. Introduction to the Ising model

Next section: 3. Introduction to Monte Carlo simulation

